Système et Réseaux : TP9, UDP et protocoles applicatifs

avril 2026 — Pierre Rousselin

Exercice 1 : Client et serveur « Incrementation Protocol » en UDP

Télécharger sur moodle les fichiers `client_udp_incp.c` et `serveur_udp_incp.c` et `Makefile`.

1. Lecture de code, côté client :

a) Comment s'assure-t-on que l'utilisateur entre bien un argument ? Dans le cas où l'utilisateur ne le fait pas, quel descripteur de fichier est utilisé pour écrire un message d'erreur et quel est le statut de sortie du processus ?

b) Quel est le type de la `sockaddr` distante ? Où est-elle stockée ? Quelles informations contient-elle et comment sont-elles renseignées ?

c) Que se passe-t-il si l'utilisateur entre autre chose qu'un entier positif ? Comment est-ce détecté ? Si besoin, `man scanf`, partie `Valeur renvoyée`.

d) Lors de l'envoi de la requête, à quoi servent les deux derniers arguments ?

e) Lors de la réception de la réponse, à quoi servent les deux derniers arguments ? Peut-on simplifier cette partie ? Si besoin, voir `man recvfrom`.

2. Lecture de code, côté serveur :

a) Quelle est l'information principale contenue dans la `sockaddr` locale du serveur ? À quoi sert-elle ?

b) Lors de la réception de la requête, à quoi servent les deux derniers arguments ? Peut-on simplifier cette partie ? Pourquoi `sl` doit être initialisée alors que `recvfrom` risque de le modifier ? Voir `man reccvfrom`

3. Essais :

a) Essayer client et serveur sur votre boucle locale (en *loopback*).

b) À plusieurs, essayer client et serveur sur deux machines distantes.

4. Mettre en place une boucle infinie chez le serveur pour qu'il puisse servir successivement plusieurs clients. Tester avec des requêtes arrivant sur la boucle locale et d'autres depuis une machine distante.

5. Nouveaux changements : un client peut maintenant faire plusieurs requêtes à la suite. Lorsque l'utilisateur entre autre chose qu'un nombre décimal (ou signale une fin de fichier), le client s'arrête après avoir fermé sa *socket*. Utiliser la valeur de retour de `scanf`. On devrait pouvoir avoir ce comportement :

```
$ ./client 127.0.0.1
Nombre à incrémenter : 45
46
Nombre à incrémenter : 79
80
Nombre à incrémenter : abc
$
```

6. Changements *majeurs* (et incompatibles) dans le protocole INCP. On passe à **INCPv2** : maintenant, le client envoie un datagramme de 6 octets. Les 4 premiers octets correspondent encore à l'entier à incrémenter, mais les 2 suivants désignent le nombre d'incrémentations voulues (en *network byte order*). Puis, si, par exemple, le serveur reçoit les entiers 42 et 4, il va envoyer les 4 datagrammes chaque fois *à 1 seconde d'intervalle*, contenant respectivement 43, 44, 45 et 46. Voici un exemple de déroulement côté client :

```
$ ./client 127.0.0.1
Nombre à incrémenter : 42
Nombre d'incrémentations : 3
43
44
45
Nombre à incrémenter : 2
Nombre d'incrémentations : 1
```

```
3
Nombre à incrémenter : aaaaaa
$
```

7. (Peut-être l'avez-vous vu venir...) Servir un client prend maintenant un temps considérable, pendant lequel on ne peut en servir d'autres ! On va donc, chez le serveur, utiliser un *thread* par client. La stratégie est la suivante :

— Le *thread* initial reçoit, avec `recvfrom` sur sa `socket` une requête et une adresse.

— Un nouveau *thread* est créé spécialement pour traiter cette requête [1]. On doit donc lui donner, d'une manière ou d'une autre les informations (lesquelles ?) nécessaire. Ce *thread* utilisera la *socket* déjà configurée pour envoyer ses messages. On travaille donc avec une seule *socket*.

— Notez que seul le *thread* initial reçoit les requêtes des clients (peut-être plusieurs par client, peu importe).

--- * ---

1. On simplifie par rapport à « la vraie vie » où on utiliserait plutôt un réservoir (*pool*) de *threads*, mais ne compliquons pas davantage...